

```

        .bind(id)
        .fetch_optional(&mut transaction).await?;
transaction.commit().await?;
Ok(())
}

```

Compile time queries

One nice thing about SQLx is that it can perform static checks against your database server. Using the `query_as` macro will achieve this. Rewriting the above `get_person` function to use the macro will look like this:

```

pub async fn get_person_macro(&self, id: i64) ->
Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as!(Person, "SELECT * from
person where id = $1", id)
        .fetch_one(&mut transaction)
        .await?;
transaction.commit().await?;
Ok(person)
}

```

The real power of it shines when you make mistakes in your SQL query. Let's change the above code to use `id_wrong` instead of `id`:

```

pub async fn get_person_macro(&self, id: i64) ->
Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as!(Person, "SELECT * from
person where id_wrong = $1", id)
        .fetch_one(&mut transaction)
        .await?;
transaction.commit().await?;
Ok(person)
}

```

Compiling the above code will give this error:

```

error: error returned from database: column "id_wrong" does
not exist
--> src/main.rs:99:22
|
99 |         let person = sqlx::query_as!(Person, "SELECT *
from person where id_wrong = $1", id)
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
|

```

= note: this error originates in the macro ``$crate::sqlx_macros::expand_query`` (in Nightly builds, run with `-Z macro-backtrace` for more info)

As the error message indicates, there is no column named `id_wrong` in your table.

Migrations

You could see above that we ran the migrations using the `sqlx-cli` tool. But in our application code, it will often be handy to run the migrations on a fresh database. The `sqlx::migration` macro is specifically meant for that. It embeds the migration into your binary and can be used to run it before the application is started. This is how your new main function will look like with migrations integration:

```

#[tokio::main]
async fn main() -> Result<> {
    let db = DBApplication::new("postgres://
postgres:postgres@localhost/sqlx-demo".into()).await?;
    sqlx::migrate!().run(&db.pool).await?;
    ...
    ...
    Ok(())
}

```

Hopefully, this article gave you a good idea of how to use SQLx for interacting with your database. If you prefer an object-relational mapping (ORM) based solution, these are some of the popular options:

- Diesel: <https://diesel.rs/>
- sea_orm: https://docs.rs/sea-orm/latest/sea_orm/
- ormx: <https://github.com/NyxCode/ormx>

Both sea_orm and ormx are built on top of SQLx. The major differentiating factor between Diesel and the other ORM based solutions is that the former isn't async IO ready yet. **END** 🐧

References

- [1] GitHub home page: <https://github.com/launchbadge/sqlx>
- [2] Field attributes documentation: <https://docs.rs/sqlx/latest/sqlx/traits/FromRow.html>
- [3] Rust types reference: <https://docs.rs/sqlx/latest/sqlx/postgres/types/index.html>

By: Sibi Prabakaran

The author is a senior engineer working at FPComplete.